

INDIVIDUAL PROJECT REPORT
ON
“CampusSphere: Registration, Rolling QR Token, Payments and Club
Ledger
for a Unified College Event and Activity-Points Platform”

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of
Technology in Computer Science & Engineering

Submitted to:



Rajasthan Technical University, Kota (Raj.)

Submitted By:

Ayushi Garg

(Roll No.: _____)

Member, Group Project CampusSphere

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering



Session: 2026-27

Department of Computer Science & Engineering

ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

CERTIFICATE

This is to certify that the individual project report entitled “**CampusSphere: Registration, Rolling QR Token, Payments and Club Ledger for a Unified College Event and Activity-Points Platform**”, submitted by **Ayushi Garg (Roll No.: _____)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering of Rajasthan Technical University, Kota**, is a record of the bonafide work carried out by her/him as a member of the project group (Govind Jangid, Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak) at **Arya College of Engineering & I.T., Jaipur** during the session 2026-27, under my guidance and supervision.

To the best of my knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri
Project Guide
Dept. of CSE, ACEIT, Jaipur

Dr. Akhil Pandey
Head of Department
Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Er. Ram Babu Buri**, for his continuous guidance, encouragement and valuable suggestions throughout the project. His insistence on clear requirements, a weekly plan and honest review helped me understand how a large software system is planned and delivered by a team.

I sincerely thank **Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T.**, for the academic support and the environment provided for this work. I am also grateful to all the faculty members of the **Department of Computer Science and Engineering** for their guidance and cooperation.

I would like to acknowledge the support of my teammates, **Govind Jangid, Garvita Jain, Ankit Yadav and Anshika Pathak**, with whom I worked on **CampusSphere**. Our discussions on interfaces between the frontend, backend and database sections, the joint review of the specification and the shared planning of the 14-week schedule greatly enriched my understanding of team-based software development. I especially thank **Govind Jangid**, our Team Leader, for coordinating the weekly meetings and the integration of all five sections.

I also thank the faculty coordinators and student leads of the fifteen clubs of Arya College, whose experience with organising events shaped the requirements of the platform.

Finally, I extend my heartfelt gratitude to my family and friends for their constant support, patience and motivation throughout the course of this project.

Ayushi Garg
(Roll No.: _____)
Computer Science & Engineering

ABSTRACT

This report presents my individual contribution to the group project **CampusSphere**, a unified web platform for college events, club treasuries, verified attendance and AICTE activity points at **Arya College of Engineering & I.T., Jaipur**. The work was divided among five students into two frontend and deployment sections (Govind Jangid and Ankit Yadav) and three backend and database sections (Garvita Jain, Ayushi Garg and Anshika Pathak).

My section is the money-and-ticket path of the platform. It covers the atomic capacity lock that prevents overbooking, solo and team registration with six-character team passcodes, the dynamic rolling QR token service based on HMAC-SHA256 with a 30-second window, Razorpay order creation, server-side signature verification and idempotent webhook processing, the immutable club ledger and treasury updates, seat-hold expiry and refund handling, the PDF certificate generator and the Event Copilot API.

The report explains the technologies and concepts used, such as atomic database operators, concurrency control, HMAC and time-based tokens, payment gateway signatures and idempotency, and double-entry style ledgers. It presents the design of each module with its algorithms, my 14-week plan, the dependencies on my teammates, the test plan with concurrency and idempotency tests, and the challenges anticipated with their planned solutions.

This is a design and planning report. The targets quoted are goals to be measured during testing and the pilot, not measured results. The work provides practical experience in concurrency-safe backend design, applied cryptography and financial correctness.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION1

- 1.1 Project Overview1
- 1.2 Team and Work Division1
- 1.3 My Role and Responsibilities1
- 1.4 Objectives of My Section2
- 1.5 Project Duration and Schedule2
- 1.6 Tools and Collaboration3

CHAPTER 2 TECHNOLOGIES AND CONCEPTS4

- 2.1 Atomic Operations in MongoDB4
- 2.2 Race Conditions and Concurrency Control4
- 2.3 Multi-Document Transactions4
- 2.4 HMAC and Time-Based Tokens4
- 2.5 Base64URL and QR Payloads4
- 2.6 Razorpay Orders, Signatures and Webhooks4
- 2.7 Idempotency5
- 2.8 Ledger Accounting5
- 2.9 Scheduled Jobs5
- 2.10 PDF and QR Generation5
- 2.11 LLM APIs and Structured Output5
- 2.12 Testing Concurrent and Financial Code5

CHAPTER 3 INDUSTRY APPLICATIONS AND RELEVANCE6

- 3.1 Ticketing and Seat Inventory6
- 3.2 Online Payments and Reconciliation6
- 3.3 Time-Based Tokens in Security6
- 3.4 Ledger Systems in Fintech6
- 3.5 Certificate Generation and Credentialing6
- 3.6 Generative AI for Content Drafting6
- 3.7 Future Scope6

CHAPTER 4 MY WORK: DESIGN AND IMPLEMENTATION APPROACH7

- 4.1 Module Structure and Boundaries7
- 4.2 Registration State Machine7
- 4.3 Atomic Capacity Lock7
- 4.4 Solo Registration API8
- 4.5 Team Registration and Passcodes8
- 4.6 Dynamic Rolling QR Token Service8
- 4.7 Razorpay Order Creation9
- 4.8 Signature Verification and Webhook Idempotency9
- 4.9 Club Ledger and Treasury10
- 4.10 Seat-Hold Expiry, Cancellation and Refunds10
- 4.11 Certificate PDF Generator10
- 4.12 Event Copilot API11
- 4.13 Error Codes and Contracts11
- 4.14 Testing Approach11
- 4.15 Anticipated Challenges and Planned Solutions11
- 4.16 Integration with Teammates12

CHAPTER 5 14-WEEK WORK PLAN13

- 5.1 My Week-by-Week Plan13
- 5.2 Team Overview and Dependencies13
- 5.3 Reporting and Review14

CHAPTER 6 LEARNING OUTCOMES AND SKILLS15

6.1 Technical Skills15

6.2 Professional Skills15

6.3 Teamwork15

6.4 Problem Solving and Debugging15

6.5 Version Control and Communication15

CHAPTER 7 CONCLUSION17

REFERENCES18

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The group project **CampusSphere** is a unified platform for college events, club treasuries, verified attendance and AICTE activity points at Arya College of Engineering & I.T., Jaipur. It replaces chat-group announcements, form-based registration, personal UPI collection, paper attendance and static certificates with one authenticated system for Students, Club Admins and the Super Admin (Dean/HOD). The platform includes a directory of fifteen clubs with dedicated treasuries, an event approval workflow, venue booking with a 30-minute buffer, atomic seat reservation, Razorpay payments credited to club ledgers, a rolling 30-second QR pass, a camera-based gate scanner, attendance-gated feedback, SHA-256-sealed certificates and an automated activity-points ledger.

The project was carried out by five students over 14 weeks under the guidance of **Er. Ram Babu Buri**. This individual report describes the part assigned to me: the backend for registration and seat reservation (atomic capacity lock, solo and team registration), the dynamic rolling QR token service, Razorpay payments with webhook idempotency, the club ledger, seat-hold expiry and refunds, the certificate PDF generator and the Event Copilot API.

1.2 Team and Work Division

The work was divided into five sections. Two members, Govind Jangid and Ankit Yadav, took the frontend and deployment. Three members, Garvita Jain, Ayushi Garg and Anshika Pathak, took the backend and the database. Table 1.1 shows the division.

Table 1.1: Division of work among the team

Member	Section	Responsibility in brief
Govind Jangid (Team Leader)	Frontend (student experience)	Student screens, rolling QR pass, checkout, certificates and points UI, AI feed, 3D twin; integration lead
Ankit Yadav	Frontend (admin consoles) and deployment	Club and Super Admin consoles, gate scanner PWA, Kanban, Docker, NGINX, CI/CD
Garvita Jain	Backend	Authentication, RBAC, clubs, events, venue buffer engine, audit, security
Ayushi Garg	Backend	Registration, capacity lock, payments, ledger, QR token service, certificate PDF
Anshika Pathak	Database and backend	Schemas, indexes, seeding, transactions, attendance, feedback, seal and points

1.3 My Role and Responsibilities

My role is the **backend developer for registration, payments and the QR token**. I was assigned the following modules:

- the atomic capacity lock and the solo registration API;
- team registration with team names, six-character passcodes, roster joining and minimum squad size checks;
- the dynamic QR token service: per-registration 256-bit seed, 30-second windows, signed payload and the `GET /registrations/:id/dynamic-qr` endpoint;
- Razorpay order creation with server-calculated amounts and server-side signature verification;
- the payment webhook with signature validation and idempotency;
- the club ledger credit and treasury balance update, together with Anshika Pathak's transaction support;
- seat-hold expiry and cancellation and refund flow;
- the vector PDF certificate generator with crest, club badge, Dean signature and QR code;
- the Event Copilot API that drafts titles, agendas, prerequisites and rules.

1.4 Objectives of My Section

- To make overbooking impossible even under heavy simultaneous registration.
- To make sure that a ticket is issued only after a payment that the server itself has verified.
- To make sure that no payment can be credited twice and no club ledger can be altered by a user.
- To create QR tokens that are cheap to verify, hard to forge and useless after 30 seconds.
- To handle abandoned payments, cancellations and refunds without leaking seats or money.
- To generate professional, printable certificates that carry the data needed for public verification.

1.5 Project Duration and Schedule

The project is planned over 14 weeks and follows the ten-phase roadmap: foundation and authentication (Weeks 1-2), clubs (Week 3), events and venues (Weeks 4-5), registration and rolling QR (Week 6), payments (Week 7), gate scanner (Week 8), feedback (Week 9), certificates and points (Week 10), AI suite (Week 11), 3D twin and deployment (Week 12), integration testing (Week 13) and pilot with documentation (Week 14). My modules are scheduled so that the capacity lock is ready in Week 4, teams in Week 5, the QR token in Week 6 and payments in Week 7, which gives the frontend and the scanner time to integrate before the festival-like pilot. My week-by-week plan is in Chapter 5.

1.6 Tools and Collaboration

I use the selected backend stack (Spring Boot 3.3 or Node.js), MongoDB with a local replica set for transaction tests, the Razorpay test mode with its sandbox keys and a webhook tunnelling tool for local testing, Postman, JUnit or Jest with a test database container, a load-testing script for parallel requests, Git and GitHub with pull requests, and the team issue board.

CHAPTER 2

TECHNOLOGIES AND CONCEPTS

This chapter explains the technologies and concepts that I use for the backend modules assigned to me.

2.1 Atomic Operations in MongoDB

An update to a single document in MongoDB is atomic: the condition and the change are applied together, and no other operation can interleave. The operator `$inc` increases a counter, and a query condition such as `registered_count < max_capacity` is evaluated in the same step. This allows a seat to be reserved without a separate read, which is the key to avoiding race conditions.

2.2 Race Conditions and Concurrency Control

A race condition occurs when the result depends on the timing of concurrent operations. In a naive implementation two students read “one seat left” and both book it. Solutions are locks, transactions or atomic conditional updates. The atomic update is the lightest and fastest because it needs no lock and no extra round trip.

2.3 Multi-Document Transactions

Some operations change several documents that must succeed or fail together, such as confirming a registration, writing a payment and crediting a ledger. MongoDB transactions on a replica set provide atomicity, consistency, isolation and durability across documents. They are used sparingly because they are slower than single-document operations.

2.4 HMAC and Time-Based Tokens

HMAC combines a secret key with a hash function (SHA-256) to give a signature that only the key holder can create. Following the idea of TOTP, the current 30-second window number is included in the signed message, so a signature is valid only for a short time. Verification recomputes the HMAC and compares it in constant time to avoid timing leaks.

2.5 Base64URL and QR Payloads

The QR payload is a small JSON object encoded with Base64URL, which is safe in URLs and QR codes. It contains ticket number, user identifier, event identifier, window number and signature. Keeping the payload short makes the QR image simple, which scans faster on phones.

2.6 Razorpay Orders, Signatures and Webhooks

The merchant creates an order for an amount in paise. After payment, the gateway returns the order identifier, payment identifier and a signature equal to HMAC-SHA256 of `order_id|payment_id` with the key `secret`. The server recomputes it. The gateway also sends signed webhooks, for example `payment.captured`, to a public URL, so the payment is recorded even if the browser is closed.

2.7 Idempotency

An operation is idempotent if repeating it has the same effect as doing it once. Because the client call and the webhook can both arrive and a webhook can be delivered several times, payment processing must be idempotent. A unique index on the payment identifier makes the database itself refuse a second record.

2.8 Ledger Accounting

A ledger is an append-only list of entries. Each entry records credit, debit and the running balance, with a reference to the payment and event. Balances are derived from, and checked against, the entries. Entries are never edited; a correction is a new entry. This gives a complete audit trail.

2.9 Scheduled Jobs

A scheduler (Spring `@Scheduled` or a cron library) runs periodic tasks such as releasing seat holds that were not paid within a timeout. Jobs must be safe to run twice and must take a lock so that only one instance runs them when the service is scaled.

2.10 PDF and QR Generation

PDFKit or iText draws text, shapes and images as vector graphics, so certificates stay sharp when printed. A QR library renders the verification URL as an image placed on the certificate.

2.11 LLM APIs and Structured Output

The Event Copilot calls a large language model with a prompt and asks for JSON output with fixed fields. The service validates the JSON against a schema, limits the length, removes unsafe content and returns it as a draft. A fallback message is returned if the model fails.

2.12 Testing Concurrent and Financial Code

Concurrency tests start many requests at once and check the invariant (registered count never exceeds capacity). Idempotency tests replay the same webhook several times. Payment tests use gateway test mode and recorded webhook fixtures.

CHAPTER 3

INDUSTRY APPLICATIONS AND RELEVANCE

3.1 Ticketing and Seat Inventory

Airlines, cinemas and event sellers all face the problem of selling a limited inventory to many people at once. The conditional atomic update used here is a lightweight form of the inventory reservation used in those systems.

3.2 Online Payments and Reconciliation

Payment gateways, banks and e-commerce companies depend on signature verification, webhooks and reconciliation. Idempotent processing is a standard requirement because networks can repeat messages.

3.3 Time-Based Tokens in Security

One-time passwords for banking apps, authenticator applications and rotating tickets all use time windows and shared secrets. The same primitive is used here for an anti-screenshot ticket.

3.4 Ledger Systems in Fintech

Wallet and accounting systems use append-only ledgers so that every balance can be explained. The club ledger is a small example with the same principles.

3.5 Certificate Generation and Credentialing

Training platforms, universities and hackathons issue digital certificates with serial numbers and QR codes. A good generator produces printable documents that carry the data needed for later verification.

3.6 Generative AI for Content Drafting

Marketing and project teams use language models to draft content that humans then edit. Using the model only for drafts with human review is a safe pattern.

3.7 Future Scope

The money path can be extended with partial refunds, promo codes, group discounts, automatic bank settlement to clubs, UPI collect requests, multi-currency, a waitlist with automatic promotion, hardware-backed key storage for seeds and anomaly detection for suspicious payment patterns.

CHAPTER 4

MY WORK: DESIGN AND IMPLEMENTATION APPROACH

This chapter describes the design of the modules assigned to me. It is based on the project specification suite and on the schedule in Chapter 5.

4.1 Module Structure and Boundaries

My modules are `registration`, `team`, `qr`, `payment`, `ledger`, `refund`, `certificate` and `copilot`. The `registration` module owns the registration documents and the seat counter. The `payment` module owns orders and payment records and calls the `ledger` module in the same transaction as the registration confirmation. The `QR` module only reads registration data. The `ledger` module exposes an internal method to append a credit, but no HTTP endpoint that changes a balance.

4.2 Registration State Machine

Table 4.1: Registration statuses

Status	Meaning	Next states
HOLD	Seat reserved while payment is in progress	CONFIRMED, EXPIRED
PENDING_TEAM	Team created but minimum squad size not reached	CONFIRMED, CANCELLED
CONFIRMED	Registered (free) or paid and verified; ticket issued	CANCELLED, REFUND_PENDING
WAITLIST	Event full; user waiting	HOLD, CANCELLED
EXPIRED	Hold timed out without payment; seat released	None
CANCELLED	Cancelled by user or admin; seat released	None
REFUND_PENDING	Refund requested or in progress	CANCELLED

Only the service can change a status; no request can send a status. This removes the risk of a student changing a `WAITLIST` registration to `CONFIRMED`.

4.3 Atomic Capacity Lock

The reservation is a single conditional update on the event document:

```
const r = await events.updateOne(  
  { _id: eventId, status: 'APPROVED',  
    'ticketing.registered_count': { $lt: maxCapacity } },  
  { $inc: { 'ticketing.registered_count': 1 } });  
if (r.matchedCount === 0) throw new CapacityExceeded();
```

Let C be the capacity. Each successful call increments the counter by one only when the counter is below C , and the check and increment cannot be separated. Therefore the counter can never exceed C and at most C calls succeed, regardless of how many requests arrive simultaneously. The remaining requests receive a sold-out answer and may join the waitlist. If the later insert of the registration document fails, a compensating `$inc: -1` runs (or both writes are placed in a transaction), so a failed write does not consume a seat permanently.

4.4 Solo Registration API

`POST /registrations/solo` takes the event identifier. The service checks that the event is APPROVED and not past, that the student has no earlier registration (the unique index on event and user is the final guard) and then reserves the seat. For a free event the registration becomes CONFIRMED immediately and a ticket number such as CS-TKT-2026-8942 and a random 256-bit seed are generated. For a paid event it becomes HOLD and the client is told to start payment. The response contains the registration identifier, ticket number if available and status.

4.5 Team Registration and Passcodes

For hackathons and esports a captain creates a team with a name and receives a passcode such as ACE-789X. `POST /teams/join` adds a student to the roster if the passcode matches an open team, the team is not full and the student has no other registration for the event. The registration is finalised when the minimum size (for example two) is reached; paid team events can be paid by the captain for all or by each member, according to the event setting. Four alphanumeric characters give $36^4 = 1,679,616$ combinations per prefix. For n teams in one event, the chance that two random codes collide is about $n^2 / (2 \times 36^4)$, which is roughly 0.3% for 100 teams, so the generator checks uniqueness within the event and retries. Joining attempts are rate limited, and a passcode is an invitation rather than a secret.

4.6 Dynamic Rolling QR Token Service

For every confirmed registration a 256-bit seed K is created with a cryptographically secure random generator and stored in the registration document. The token is derived as

$$T = \text{floor}(t / 30) \quad (1)$$

$$\text{sig} = \text{HMAC-SHA256}(K, T || \text{ticket_number} || \text{user_id}) \quad (2)$$

`GET /registrations/:id/dynamic-qr` is available to the ticket owner and the Club Admin of the organising club. It returns the Base64URL payload `{ tkt, uid, eid, t,`

`sig }`, the number of seconds for which it is valid and the expiry time, for example `valid_for_seconds: 24` when requested six seconds into the window. The frontend refreshes the token before expiry. The seed itself is never returned by the API.

The validation routine used by the scan API of Anshika Pathak performs the following steps: decode the payload; check that the event matches; accept windows T-1, T and T+1 to tolerate clock difference; reject any timestamp difference above 45 seconds; recompute the HMAC with the stored seed and compare in constant time. A random 256-bit signature has a 2^{-256} chance of being guessed. Since a scanner window accepts three token windows, a token could be accepted for up to 90 seconds; using the stricter 45-second rule as the single normative limit keeps the replay exposure small, and the single-use attendance check removes the value of a replay.

4.7 Razorpay Order Creation

`POST /payments/create-order` takes the event identifier and registration type. The server loads the event, computes the amount in paise from its own ticket price (for example INR 250 = 25,000 paise), verifies that the user has a HOLD registration, creates the Razorpay order with the receipt set to the registration identifier and returns the order identifier, amount, currency, public key identifier and club name. The client can neither supply nor change the amount.

4.8 Signature Verification and Webhook Idempotency

`POST /payments/verify-signature` receives the order identifier, payment identifier and signature. The server computes `HMAC_SHA256(order_id + '|' + payment_id, secret)` and compares. On mismatch the request is rejected and a fraud entry is logged. The webhook endpoint accepts `payment.captured` events only after validating the `X-Razorpay-Signature` header against the webhook secret. The common routine (Figure 4.1) checks whether the payment identifier already exists, and if so returns 200 without changes. Otherwise a single transaction confirms the registration, generates the ticket and seed, writes the payment with the club identifier, appends the ledger credit and updates the club balance.

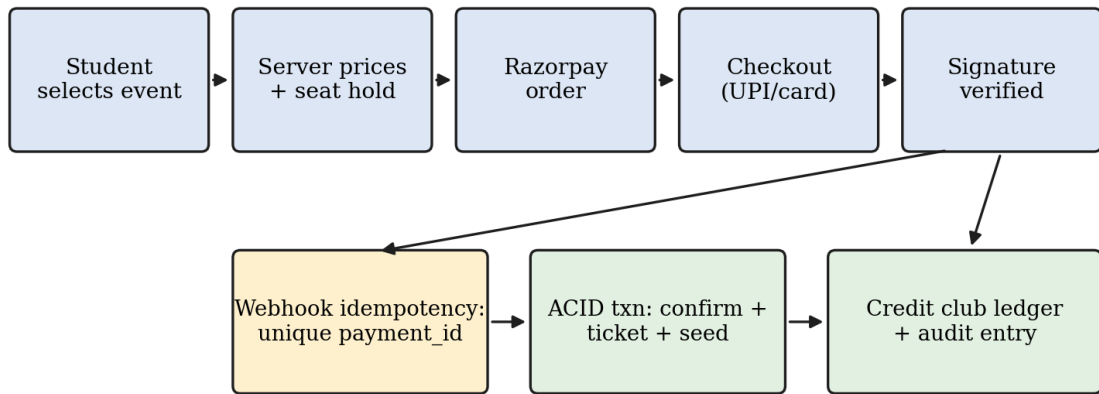


Figure 4.1: Payment verification and ledger settlement flow

The two arrival paths (client verification and webhook) can run at the same time. The unique index on the payment identifier is the final arbiter: the second insert fails and its handler returns success without crediting again. Table 4.2 lists the failure cases and their handling.

Table 4.2: Payment failure cases

Case	Handling
Signature mismatch	Reject, log fraud attempt, no ticket
Webhook delivered several times	Idempotent no-op after the first
Client closes browser after paying	Webhook confirms the registration
Payment captured after hold expired	Confirm if seat can be re-reserved; otherwise automatic refund
Gateway timeout	Registration stays HOLD; user can retry; reconciliation job checks status
Amount different from expected	Reject and flag for review

4.9 Club Ledger and Treasury

Each verified payment creates one ledger entry for the organising club with type `TICKET_SALE`, credit amount (net of any gateway fee), running balance and remarks such as the ticket number. The club's `available_balance` and `total_revenue` are updated in the same transaction. Refunds create a debit entry. Entries are append-only, and there is no HTTP endpoint that changes a balance. A nightly job recomputes each club's balance from its entries and raises an alert if they differ.

Table 4.3: Ledger entry types

Type	Effect
<code>TICKET_SALE</code>	Credit equal to net amount of a verified payment
<code>REFUND</code>	Debit when a refund is completed
<code>SETTLEMENT</code>	Debit when the college accounts office pays out to the club

Type	Effect
ADJUSTMENT	Credit or debit entered only by Super Admin with a reason, always audited

4.10 Seat-Hold Expiry, Cancellation and Refunds

A scheduled job runs every minute and finds HOLD registrations older than the timeout (for example ten minutes). For each, it marks the registration EXPIRED and decrements the event counter. A cancellation by the student before a cut-off time releases the seat and, if paid, creates a REFUND_PENDING entry that calls the gateway refund API; the ledger debit is written when the refund is confirmed. The job takes a lock so that only one instance runs, and each action is safe to repeat.

4.11 Certificate PDF Generator

After an event is COMPLETED, the certificate service receives the verified attendee list and the seal and serial number from the module of Anshika Pathak. It draws a vector PDF with the Arya College crest, club badge, student name, event title, date, points, serial such as CS-ARYA-2026-HACK-0142, the Dean's signature image and a QR code for the public verification URL. Files are stored in S3 or Cloundinary and linked in the certificate record. Generation runs as a background batch and is idempotent: a certificate is not generated twice for the same registration.

4.12 Event Copilot API

POST /copilot/event-draft accepts a short prompt and optional category. The service builds a fixed prompt that asks the model for JSON containing a title, summary, agenda items with times, prerequisites and rules. It validates the response against a schema, trims lengths, filters unsafe content and returns it as a draft. Requests are limited per user, and the result is stored only if the Club Admin saves the event. The Super Admin still reviews the final proposal.

4.13 Error Codes and Contracts

Typical codes are CAPACITY_EXCEEDED (409, with waitlist information), ALREADY_REGISTERED (409), EVENT_NOT_OPEN (400), TEAM_FULL (409), INVALID_TEAM_CODE (404), PAYMENT_SIGNATURE_INVALID (400), QR_EXPIRED and QR_INVALID (used by the scan API). Each is documented in the OpenAPI file.

4.14 Testing Approach

Table 4.4: Test plan for my section

Level	Tool	Examples
Unit	JUnit / Jest	Token derivation at window edges; passcode uniqueness; amount calculation
Concurrency	Parallel request script	500 requests for capacity 400 give exactly 400 confirmations
Idempotency	Replay fixtures	Same webhook five times gives one payment and one ledger credit
Integration	Gateway test mode	Create order, pay with test card, verify, ticket issued
Security	Negative tests	Tampered signature rejected; client-supplied amount ignored
Job tests	Controllable clock	Hold expires after timeout and seat is released once
Reconciliation	Nightly check	Balance equals sum of entries

4.15 Anticipated Challenges and Planned Solutions

Table 4.5: Challenges and solutions

Challenge	Planned solution
Overbooking under flash registration	Conditional atomic update; concurrency test in CI
Leaked seats after failures	Transaction or compensating decrement; hold expiry job
Double credit from duplicate callbacks	Unique payment index; idempotent handlers
Clock differences affecting QR	Three-window tolerance with 45-second cap; server time in responses
Seed exposure to scanners	Seeds stay on the server; per-event derived keys for offline mode
LLM returning invalid output	Schema validation, retries and fallback message

4.16 Integration with Teammates

Govind Jangid builds the registration, pass and checkout screens on my APIs and Ankit Yadav builds the roster, finance console and scanner. Garvita Jain provides the event states and roles that my services check. Anshika Pathak provides the transaction helper, the attendance and scan API that uses my validation routine, and the certificate seal and points that my generator prints. We agree the token payload and error codes in Week 6 and record them in the interface table.

CHAPTER 5

14-WEEK WORK PLAN

5.1 My Week-by-Week Plan

Table 5.1 gives my plan for the 14 weeks together with the expected deliverable of each week. The weeks follow the ten phases of the roadmap.

Table 5.1: My 14-week plan

Week	Phase	My tasks	Deliverable / check
1	Setup	Study Razorpay and seat-lock design; draft registration and payment schemas	Design notes reviewed; flows drawn
2	1	Registration and payment collections with unique indexes; DTOs	Unique indexes created; DTOs reviewed
3	2	Treasury and ledger models; running-balance logic	Ledger entry and running balance tests pass
4	3	Atomic capacity lock and solo registration API with unit tests	500 parallel requests never exceed capacity
5	3	Team registration, passcode generator, roster join API	Team joins and passcode uniqueness tested
6	4	Dynamic QR token service (seed, time window, signature)	Token valid in window and rejected after
7	5	Create-order, signature verification, webhook idempotency	Test payment gives ticket and ledger credit
8	6	Shared token-validation library; clock-skew tests	Scanner and backend accept the same token
9	7	Seat-hold expiry job; cancellation and refund flow	Seat released once on expiry; refund works
10	8	Certificate PDF generator (crest, QR, signature)	Printable certificate with QR
11	9	Event Copilot API (prompts and validation)	Valid JSON drafts returned
12	10	Payment end-to-end tests	Payment end-to-end tests pass in CI
13	Test	Concurrency and idempotency stress tests	Stress tests pass; no double credit
14	Pilot	Payment reconciliation report	Reconciliation report matches ledgers

5.2 Team Overview and Dependencies

Figure 5.1 shows the whole team schedule. In every week each member has one main deliverable, and the weekly meeting reviews all five. The dependencies between my tasks and the work of the other members are listed in Table 5.2.

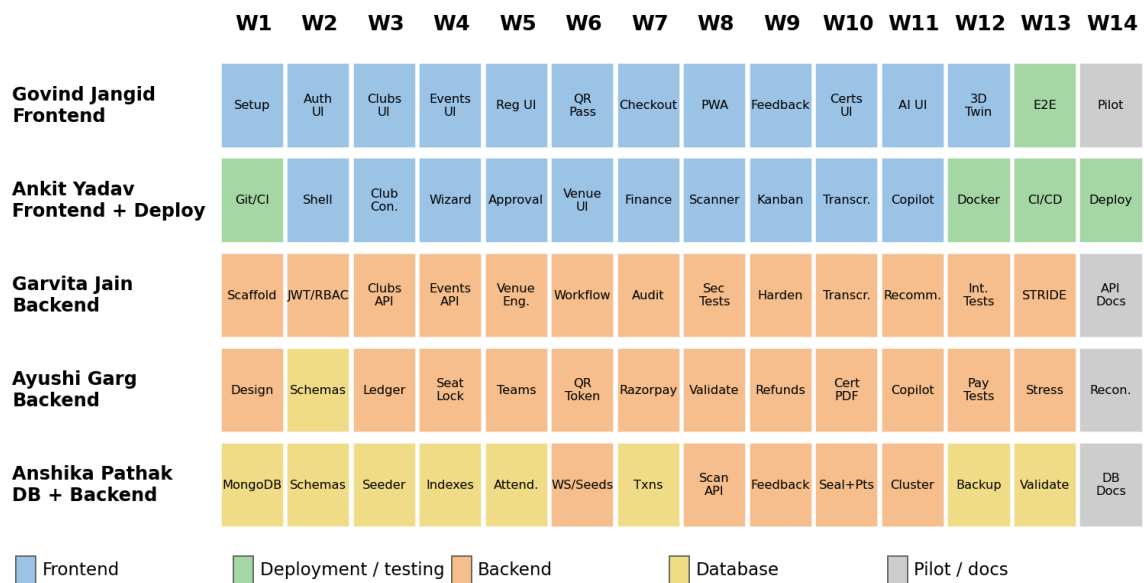


Figure 5.1: 14-week work distribution of the five members

Table 5.2: Dependencies of my weekly tasks on teammates

Week	My task	Depends on / provides to
2	Schemas and indexes	Anshika Pathak: schema review; Garvita Jain: error envelope
3	Ledger models	Anshika Pathak: seeder and indexes
4-5	Capacity lock and teams	Garvita Jain: APPROVED event state; Govind Jangid: registration UI
6	QR token	Govind Jangid: pass screen; Ankit Yadav and Anshika Pathak: scanner and scan API
7	Payments	Govind Jangid: checkout; Anshika Pathak: transaction helper; Ankit Yadav: finance console
8	Validation library	Anshika Pathak: verify-scan API
10	Certificates	Anshika Pathak: seal and serial; Govind Jangid: wallet
11	Copilot API	Ankit Yadav: wizard panel
12-14	Tests and reconciliation	Ankit Yadav: CI; Garvita Jain: audit entries

5.3 Reporting and Review

Every Friday I give the Team Leader a short status note listing completed tasks, tasks in progress, blockers and changes to interfaces. The guide reviews progress at the end of key milestones (Weeks 2, 5, 7, 9, 11, 13 and 14). A slip is first handled by re-ordering tasks within the week and then, if needed, by moving work between sections in agreement with the affected members.

CHAPTER 6

LEARNING OUTCOMES AND SKILLS

6.1 Technical Skills

- Designing concurrency-safe inventory reservation with atomic conditional updates.
- Using MongoDB transactions where several documents must change together.
- Implementing HMAC-based time-windowed tokens and constant-time verification.
- Integrating a payment gateway with server-side signatures and idempotent webhooks.
- Designing an append-only ledger with reconciliation.
- Writing scheduled jobs that are safe to repeat and to run on several instances.
- Generating vector PDFs with QR codes and validating LLM output against a schema.
- Writing concurrency, idempotency and negative security tests.

6.2 Professional Skills

- Reading and applying a formal specification suite (PRD, SRS, TRD, ERD, API specification).
- Contract-first collaboration between the frontend, backend and database sections.
- Planning work in weekly deliverables and tracking progress with an issue board.
- Writing clear documentation and status reports.

6.3 Teamwork

My modules sit between the frontend, the scanner and the database, so I learned to agree on exact payload formats and error codes with each neighbour before coding. Working with Anshika Pathak on transactions and with Garvita Jain on event states showed me how one rule, such as “only APPROVED events can be booked”, must be consistent in three modules.

6.4 Problem Solving and Debugging

Problems in concurrent and financial code are rare but expensive, so reasoning about invariants matters more than trial and error. I plan to state each invariant explicitly (for example, registered count never exceeds capacity; one payment gives one credit), write a test that tries to break it, and fix the design if the test succeeds in breaking it.

6.5 Version Control and Communication

Short-lived branches, small pull requests and meaningful commit messages make collaboration smoother and keep the main branch always deployable. Clear and timely communication with the other four members, including early notice of changes to an interface, is as important as writing good code.

CHAPTER 7

CONCLUSION

This individual report presented my part in the CampusSphere project: registration and seat reservation, the rolling QR token, payments and the club ledger, seat-hold and refund handling, the certificate PDF generator and the Event Copilot API. The work is divided among five members into two frontend and deployment sections and three backend and database sections, planned over 14 weeks following the ten-phase roadmap.

My modules protect the points where trust and money meet: a seat can be reserved only if one is free, a ticket exists only after a verified payment, a payment is credited once and only to the organising club, and a pass is useful for only a short time. The design rests on atomic updates, signatures and idempotency, which are simple ideas that are easy to test.

This report documents the design and the plan of my work. The performance and correctness targets are to be verified in Weeks 12 to 14 and in the pilot. The experience has given me a strong base in concurrency, applied cryptography and payment integration for professional backend work.

REFERENCES

- [1]AICTE, “Mandatory extracurricular activity points for B.Tech degree,” All India Council for Technical Education, New Delhi, and Rajasthan Technical University, Kota.
- [2]CampusSphere Project Team, “Product Requirements Document (PRD), v1.0.0-PROD,” Arya College of Engineering & I.T., Jaipur, 2026.
- [3]CampusSphere Project Team, “Software Requirements Specification (SRS), IEEE Std 830 format, v1.0.0-PROD,” 2026.
- [4]CampusSphere Project Team, “Technical Requirements Document (TRD), v1.0.0-PROD,” 2026.
- [5]CampusSphere Project Team, “REST API Specification, v1.0.0-PROD,” 2026.
- [6]CampusSphere Project Team, “Security Architecture and Fraud Prevention Specification, v1.0.0-PROD,” 2026.
- [7]H. Krawczyk, M. Bellare and R. Canetti, “HMAC: Keyed-hashing for message authentication,” IETF RFC 2104, Feb. 1997.
- [8]D. M’Raihi, S. Machani, M. Pei and J. Rydell, “TOTP: Time-based one-time password algorithm,” IETF RFC 6238, May 2011.
- [9]National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.
- [10]MongoDB Inc., “Transactions and atomicity,” MongoDB Manual 7.0. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [11]Razorpay, “Payment gateway and webhook documentation.” [Online]. Available: <https://razorpay.com/docs/>
- [12]M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [13]A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.